Evaluating LLM's Code Reading Abilities in Big Data Contexts using Metamorphic Testing

1st Ziyu Li
University of Sheffield
zli311@sheffield.ac.uk

2nd Zhendu Li
College of Systems Engineering
National University of Defense Technology
lizhendu@163.com

3th Kaiming Xiao*
Laboratory for Big Data and Decision
National University of Defense Technology
kmxiao@nudt.edu.cn

4th Xuan Li
Laboratory for Big Data and Decision
National University of Defense Technology
xlhunan@126.com

2023 9th International Conference on Big Data and Information Analytics (BigDIA) | 979-8-3503-3007-6/23/$31.00 ©2023 IEEE | DOI: 10.1109/BigDIA60676.2023.10429345

***Abstract*—With the explosive growth of Big Data, understanding complex data-centering algorithms and software has become essential. Large Language Models (LLMs) especially ChatGPT models have been increasingly deployed in Big Data environments to improve workflow i n v arious t asks, i ncluding c ode reading. The current testing method on LLMs' code reading ability focuses more on code structural understanding and sentiment understanding, all tests are conducted on different prompts with different assumptions. This paper analyzes current LLM code-reading testing methods and presents an innovative evaluation of Metamorphic Testing to evaluate LLMs' code-reading abilities. We proposed two new metamorphic relations specified f or code reading challenges and evaluated the ChatGPT-3.5 on its code understanding capabilities. Our study offers insights into LLMs' capabilities to correctly understand diverse code bases and maintain validity.**

I. Introduction

The field o f a rtificial in telligence (A I), pa rticularly within large-scale computational environments, has seen notable advancements with the introduction of Large Language Models (LLMs). Notably, the Generative Pretrained Transformers (GPT), demonstrated especially by ChatGPT-3.5, have set benchmarks in their ability to generate human-like responses, code, and other text formats. These developments have prompted the creation of similar models by various organisations, such as PaLM [1] and LLaMA [2].

With the rapid expansion of big data, LLMs are increasingly being utilised to help understand and generate complicated data algorithms and software. Ensuring the accuracy and reliability of these models becomes essential, especially when minor coding errors can lead to significant software vulnerabilities or operational issues. Evaluations on datasets like HumanEval [3] and MBPP [4] have highlighted these inconsistencies and the areas where LLMs could be improved.

Considering the broad application potential of these models, particularly in big data environments, it becomes critical to assess their understanding of diverse inputs and their ability to identify software issues. This research aims to identify potential vulnerabilities in these models in code reading ability to enhance the reliability and robustness of LLMs.

Metamorphic Testing [5] is a form of software testing, in contexts where traditional test oracles (a mechanism used to determine whether a test has passed or failed) are hard to define or apply, or the cost of testing is expensive. Metamorphic relations are properties that define how the output of a system should change based on a certain transformation of its input and will be used to check the correctness of a program without knowing the expected outputs. Testing the output of LLMs using the metamorphic test will avoid the Oracle issue and the cost of the LLM deploying fee. This will offer an innovative approach to assess the code understanding capabilities of LLMs. Applying this method to the LLMs will give us an opportunity to evaluate the robustness and reliability of LLMs in understanding and interpreting code at a reasonable cost.

The objectives of this research include:

- Exploring Current LLM Evaluation Techniques: Initially, we aim to examine the methods currently in use to assess the performance of LLMs, including identifying the most appropriate datasets and evaluation techniques.
- Designing Metamorphic Testing Experiments: The primary goal here is to design experiments that use Metamorphic Testing to evaluate the code-understanding ability of LLMs while avoiding excessive use of computing resources and providing validity.
- Analysing Performance Across Different LLMs: We will conduct experiments with different versions of LLMs to understand their responses and capabilities in addressing code reading challenges within Big Data.
- Evaluating the Results: Our analysis will focus on comparing the performance of different LLMs and understanding the strengths and weaknesses of each in Metamorphic Testing within Big Data contexts.

979-8-3503-3007-6/23/$31.00 ©2023 IEEE

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 01:36:35 UTC from IEEE Xplore. Restrictions apply.

The report is structured as follows. Section II gives an introduction to LLMs and the basics of Metamorphic Testing, highlighting its importance in Big Data studies. Section III explains the experiments in detail, covering the datasets used, the method of prompt modification, and code modification. Section IV report and discusses our results and major findings, comparing the performance of different LLMs using metamorphic testing. Section V introduces previous research, showing where our study fits in the larger academic discussion and what makes it unique. Section VI wraps up our study's main points, presents our conclusions, and suggests topics for future research.

II. Background

A. Large Language Models

Large Language Models (LLMs), such as Generative Pre-trained Transformer (GPT) models, are neural network architectures designed to understand and generate text based on context. They are trained semi-supervised using both formatted data and unformatted data and are capable of performing a wide variety of natural language processing tasks, including text generation [6, 7], translation [1, 8], summarization [9, 10], question answering [11, 12], and code generation and comprehension [13, 14]. text generation [6, 7].

Over recent years, Large Language Models (LLMs) have become a key component in Natural Language Processing (NLP) research and applications. Plus the architectures of these models have been evolving, both their size and capabilities expanded exponentially, and remarkable progress has been made by those models across multiple tasks.

The proposal of seq2seq by Google [15], instead of using an older statistical machine translation approach, they started implementing the latest neural network into their machine translation task. By combining the two critical approaches, the attention mechanism [16] (a technique that allows the model to focus on specific parts of the input sequence when producing an output) and LSTM [17] network (a type of recurrent neural network designed to remember long-term dependencies), they propose the transformer structure [18], a cornerstone in the realm of Natural Language Processing (NLP).

Initial models such as GPT [19] and BERT [20] use transformers to effectively capture long-range dependencies in text. Their success started a broad interest in unsupervised and semi-supervised learning approaches(a training approach with some and no supervision to save marking and dataset making), to save the cost of making a dataset and to increase the size of the model. This was evident with the development of GPT-2 [21], a model with 1.5 billion parameters, known for its capacity to generate long coherent and diverse text.

Simultaneously, BERT [20] highlighted the advantages of bidirectional training for its ability to context in understanding text. This led to the development of various specialized models, such as RoBERTa [22] and DistilBERT [23], each addressing specific challenges or optimizing performance in the NLP domain.

With the introduction of GPT-3 [8], a model enriched with 175 billion parameters, focusing more on the attention mechanism, shows its advancements in few-shot and zero-shot learning. GPT-3's effectiveness in understanding natural language is magnificent. Then along with Codex [3] and GPT2, OpenAI officially released ChatGPT as a chatbot.

Finally, the improved version of ChatGPT-3.5 and GPT-4 [24] released, trained extensively on a diverse set of aligned image-text pairs, strengthening its ability in the field of visual understanding and reasoning. And it is way more reliable and creative according to OpenAI.

B. Implications for Big Data

As the complexity and capacities of LLMs augmented, their relevance and applicability in Big Data contexts became more pronounced. Big Data, characterized by its volume, variety, velocity, and veracity [25], presents challenges that demand complicated tools for interpretation and analysis.

LLMs, particularly those tailored for code comprehension and generation like Codex, offer promising methods to understand intricate algorithms and software pivotal to Big Data. Ensuring the reliability of these LLMs is paramount, especially considering the potential consequences of erroneous code interpretation or the generation of vast amounts of data.

In summation, the combination between LLMs and Big Data appears to be not only inevitable but also essential. The prowess of LLMs, coupled with the vastness of Big Data, marks the beginning of more informed decision-making and complicated data analytics.

C. Prompt Engineering in LLMs

Prompt engineering refines input prompts, enabling language models to give precise outputs tailored to context. Prompt engineering is about crafting input texts to elicit accurate and relevant responses from a model. Fig1 displays an example of a prompt for code generation tasks using LLMs.

User:

Please generate the appropriate code based on the following description.
Description: C equals A plus B;

ChatGPT:

Certainly! Here's the Python code for that:

```
C = A + B
```

Fig. 1. An illustrative interaction with an LLM

Prompts, in essence, are more than mere questions; they serve as pivotal tools for maximizing a model's efficacy. The pioneering work by Laria Reynolds and Kyle McDonell [26] offers profound insights into this domain's intricacy. Their investigations divulged that conventional methods, notably the shot-based technique [19], may not always extract the model's

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 01:36:35 UTC from IEEE Xplore. Restrictions apply.

utmost potential. Interestingly, they established that adeptly crafted 0-shot prompts could, in specific instances, surpass the outcomes of few-shot prompts.

Further accentuating the significance of prompt engineering is the study by J Wei [27]. Given that models such as GPT-3 are nurtured on extensive human-crafted data, they inherently mirror human communication dynamics. Thus, prompts that furnish a series of rational intermediary steps are invaluable for LLMs' comprehension. This insight ushered in an anthropomorphic perspective towards prompt engineering, conceiving the model as a virtual human correspondent and instituting a coherent communicative framework.

D. Metamorphic Testing

Metamorphic testing is an approach to software testing that hinges upon the principle of metamorphic relations, which are necessary properties of the target function under test, even if the exact expected outputs are unknown [5]. This technique addresses the challenges in scenarios where test oracles (i.e., mechanisms to determine the correctness of the output) are unavailable or too expensive to use. Instead of verifying the correctness of individual outputs, metamorphic testing checks if the relation between multiple inputs and their corresponding outputs adheres to the expected metamorphic relations.

Metamorphic relation is a property that specifies how the input and output of a program should be related under certain transformations. For example, a metamorphic relation for a sorting program is that reversing the order of the input and output should yield the same result. The primary goal of metamorphic testing is to detect faults that are not revealed by the original test cases [5]. If the program fails to satisfy a metamorphic relation, it indicates a potential defect in the program.

In the context of Code LLMs in our experiment, metamorphic testing provides a novel way to evaluate the machine learning models [28, 29, 30]. Differently, our approach evaluates models' ability to generate and process code. By applying metamorphic relations to the code snippets and descriptions given to the models, we can create new test cases that challenge the models' understanding and generation of code. For instance, when we reverse the order of statements in a code snippet, we expect the model to generate a description that reflects the change in execution order. This evaluation is important because it helps us discover the limitations and weaknesses of the models when they encounter slightly modified or different codes. Given that real-world coding often involves adapting and modifying existing code, understanding how Code LLMs handle these scenarios is essential for their practical use.

III. Approach

In Chapter II, we introduced fundamental concepts and background. We'll further explore the core of our study. We identify two essential Metamorphic Relations (MRs) as the base of our experiment."

A. Metamorphic Relations

In our experiments, we investigate two primary metamorphic relations:

- MR1: Does altering the prompt without changing the description and code affect the LLMs' ability to understand the code?
- MR2: Does changing values or relational operators within the code impact the LLMs' ability to understand the code?

MR1 focuses on the effect of prompt modifications on Large Language Models (LLMs), such as GPT-3.5, without altering the code's description and content. This MR aims to determine if mere prompt changes can impact the LLMs' capability to comprehend the code's intent and purpose.

MR2 investigates how alterations in the values or relational operators within the code affect the LLMs' understanding. This MR seeks to assess if such intrinsic modifications, which directly change parts of the code but not its overall structure, can influence the LLMs' ability to accurately grasp the code's functionality.

B. Datasets

For our evaluation, we utilized the widely acknowledged open-source dataset, HumanEval [3]. The structure of this dataset is critical to our prompt design and evaluation setup detailed as follows.

The HumanEval dataset is a handwritten evaluation dataset proposed by OpenAI to assess the code generation ability of code LLMs. It comprises 164 Python programming problems that span multiple facets including language understanding, algorithms, and basic mathematics. Each entry is accompanied by a solution to the question and its test data. This dataset has been instrumental in gauging the accuracy of the code produced by an array of Code LLMs. Data examples in Table I.

TABLE I
SAMPLE REPRESENTATION OF THE DATASET

task_id	prompt	canonical_solution
"PYTHON/0"	"Check if in given li..."	" for idx, elem in enum..."
"PYTHON/1"	"Input to this functio..."	" result = []\n current_s..."
"PYTHON/2"	"Given a positive flo..."	" return number % 1.0\..."

C. Evaluation Subjects

In our evaluation, we chose two specific versions of GPT-3.5 to ensure a nuanced understanding of their capabilities and limitations, especially in the context of code-related tasks. The chosen models are the previous version and the improved version, thereby providing a comprehensive perspective on our evaluation process, and a chance to see the progress of the model's improvement. Below, we detail the primary subjects of our evaluation:

1) GPT-3.5-0301: GPT-3.5-0301 is a particular configuration of the GPT-3.5 model. With a significant parameter count and comprehensive training data, this version showcased profound capabilities in understanding and generating human-like text, including the generation of code.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 01:36:35 UTC from IEEE Xplore. Restrictions apply.

2) GPT-3.5-0613: GPT-3.5-0613 is the 13th June update of GPT-3.5. This variant aimed to refine and improve upon the capabilities of its prompt handling, providing enhanced code understanding and generation abilities. According to [31], this updated version is more controllable with the system message.

The LLMs chosen for this evaluation were based on specific criteria, such as:

- **Relevance**: We opted for models that have garnered significant attention been recognized in the research community, and are widely employed in real-world scenarios.
- **Performance**: Our interest was piqued by models demonstrating superior capabilities, be it in general tasks or specific domains.
- **Ability**: An inherent capability for code generation and understanding is essential for our experiments; thus, our chosen models embody these functions.

In summation, focusing on GPT-3.5-0301 and GPT-3.5-0613 provides valuable insights into the capabilities of LLMs in their various configurations, guiding future research and development in this field.

D. Methodology

Here is the overall diagram of our experiment, including Code Modification, Prompt Modification, and Result Evaluation. Providing a clear explanation of our experiment Fig 2. One of the foundational pillars of our experiment was the consistent modification of our data, adhering strictly to the principles we elucidated below.

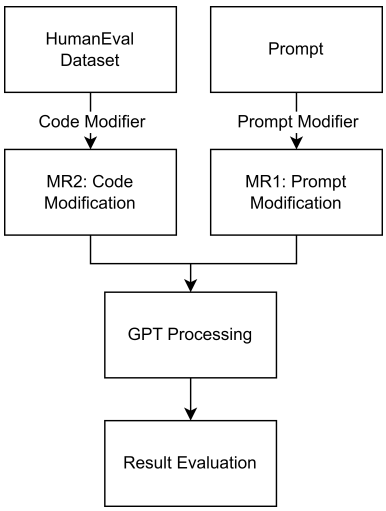


Fig. 2. Experiment Step

The output from the Large Language Models (LLMs) will be limited to two distinct numerical responses: '1' and '2'. In particular, a '1' signifies a complete alignment between the code and its description. Conversely, '2' could either mean a partial alignment or a total misalignment.

1) Prompt Modification: For prompt modification, the modifications are applied to the initial prompts or questions posed to the LLMs. The prompt was categorized into three core sections: Role Definition(brown), Task Declaration (teal), and Output Declaration (violet) as depicted in Fig3. Our evaluation method permits modifications only to the Role Definition and Task Declaration. The Task Declaration is the portion where the user delineates the specific task or question for the model, ensuring clarity and context. The Role Definition, on the other hand, sets the tone or context for the task, establishing the perspective from which the LLM should operate.

User:
You are a code-reading assistant.
I'm presenting you with a code, tell me the meaning of the code.
Code: CODE. Evaluate the code.
Respond with: 1 - If the code aligns with the description; 2 - In all other scenarios.

Fig. 3. Prototype of the Prompt Used in Evaluations

By removing either or both of the Role Definition and Task Declaration parts, can we evaluate how minor alterations in the input prompt can influence the model's response, especially when generating or understanding code.

2) Code Modification: The creation of modified code fragments is realized by purposefully introducing subtle changes to the original code. These alterations encompass operators such as arithmetical and relational operator replacement, value adjustments, and statement deletion. The primary objective of these modifications is to maintain the overall structure and semantics of the code while subtly impacting its functionality.

Our focus was on introducing numerical and logical tweaks to the code. Within the 'NUM' category, we manipulated the numerical constants present in the code. In the 'LOG' category, we shifted the logical operators, such as altering a '+' to a '-', and vice versa. Numerical adjustments are targeted at testing the model's ability to retain logic with different values, while logical changes evaluate the model's resilience to foundational shifts in code logic.

3) LLM Execution with Prompt Engineering: Prompt Engineering is a technique employed to enhance the performance of large language models (LLM) like GPT. The approach revolves around the design and structure of the input prompts given to the model. By refining the format in which questions are articulated, structuring the input in specific formats, and occasionally integrating examples or frameworks for thought, the overall quality, accuracy, and relevance of the model's output can be improved. When applying modifications to the target code, prompt engineering is necessary to improve the model's accuracy and comprehension of the code.

Our prompts are carefully crafted, inspired by common methods used for testing Code LLMs on the HumanEval Dataset [3]. The specific prompt we employ is as follows in Fig 4.

This prompt serves multiple purposes:

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 01:36:35 UTC from IEEE Xplore. Restrictions apply.

- **Role Definition**: By designating the model as a "code reading assistant," we set the context and role for the model, guiding its behaviour.
- **Structured Input**: The prompt provides a structured format for the input, separating the code description and the code itself, which aids in better comprehension by the model.
- **Clear Evaluation Criteria**: The prompt specifies the evaluation criteria explicitly, asking the model to respond with a single number that represents the code's alignment with the given description.

User:
You are a code-reading assistant. I'm presenting you with a code, tell me the meaning of the code.
Code: CODE.

GPT respond: code description

User:
Description: DESCRIPTION;
Evaluate the code in relation to the description.
Respond with:
1 - If the code, both in function and syntax, entirely aligns with the description;
2 - In all other scenarios (for instance, minor syntactical or logical discrepancies or if the code doesn't match the description at all).
Your response should be a solitary number: '1' or '2'.

GPT respond: final result ('1' or '2')

Fig. 4. Specific Prompt Employed

By employing this carefully designed prompt, we aim to maximize the model's understanding and evaluation of modified code snippets, thereby providing a more accurate measure of its code comprehension abilities.

4) Accuracy Comparison: Our primary approach to evaluating the LLM's understanding of code revolves around comparing the accuracy of its responses to original prompts versus modified ones. This method allows us to ascertain the model's adaptability and sensitivity to subtle changes.

We compare accuracy(both original and modified) across models to provide a direct measure of the model's performance. A high accuracy in the face of modified prompts indicates the model's robustness and capability to maintain comprehension even with subtle changes in input. As for code modification, a low accuracy indicates the models can not deal with subtle changes in the code.

Additionally, by using different versions of models, such as various iterations of GPT-3.5, we can observe the progress and advancements made in the model's ability to handle code. This comparative approach not only reveals the strengths and weaknesses of each version but also offers insights into the evolutionary directions of LLMs in terms of code understanding.

In conclusion, through a meticulous accuracy comparison between responses to original and modified prompts across diverse versions of the LLM, our evaluation offers a detailed perspective on the model's proficiency, adaptability, and areas that require further refinement.

IV. Experiment Result

Building on the methodology detailed in Chapter III, our experimental structure was carefully designed to ensure the robustness and credibility of our findings.

A. MR1: Prompt Modifications Impact

In this MR, we delved into exploring the influence of diverse Prompt Modifications on the code understanding ability of LLMs. Specifically, GPT-3.5's results highlighted that its code understanding ability noticeably fluctuates based on the type of prompt alteration. This section will furnish a comprehensive analysis of these observations.

1) How Performance of Models Varies in Different Prompt Modifications: The data suggests notable disparities in the efficacy of GPT-3.5 when subjected to different Prompt Modifications, as delineated in Fig3. For instance, while the GPT-3.5-0301 seems to be stable on all prompt modification types, showcased commendable precision in code understanding modifications in Task Declaration. The GPT-3.5-0613 version shows highly variable data when using different types of modified prompts and was more sensitive to minimal prompt alterations. The specifics of these behaviours, alongside their implications on the model's code comprehension capabilities, will be further elaborated on in the subsequent sections.

TABLE II
Impact of Prompt Modification on Accuracy.

Modification Target[a]	GPT-3.5-0301 (%)	GPT-3.5-0613 (%)
None	96.29	96.29
Task Declaration	96.29	98.76
Role Definition	95.06	86.41
Both	95.06	91.35

[a]Modification Target is the part we remove from the initial prompt. As demonstrated in Fig 3.

Task Declaration: GPT-3.5-0613's high performance across various modifications can be attributed to its refined architecture or training data, which significantly provide better contextual understanding and adaptability. GPT-3.5-0301 on the other hand is not sensitive to the modification in Task Declaration and remains high performance despite the prompt change.

Role Definition: Both models showcase commendable performance under Role Definition alterations. The low performance of GPT-3.5-0613 is unexpected and implies enhancements in model self-identification in this version. The more

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 01:36:35 UTC from IEEE Xplore. Restrictions apply.

prompt about Role Definition can significantly improve the accuracy. The GPT-3.5-0301 model, while slightly trailing, signifies a promising ability to correlate role with context.

Role Definition & Task Declaration: The result change in GPT-3.5-0301 is subtle, and the change in accuracy is less than 2%, it is good to see the robustness in GPT-3.5-0301 on prompt modification, yet this also means the subtle change in the prompt is less noticeable to this version of GPT3.5. As for GPT-3.5-0613, The prompt can significantly influence the accuracy of code understanding, this indicates the model relies highly on the prompt and is less robust to prompt change, yet this means the model is more user-friendly and can react more sensitively to the user's input.

GPT-3.5-0301: The GPT-3.5-0613 version clearly demonstrates superior adaptability in managing simultaneous prompt modifications, signifying iterative enhancements in model design. Yet this can also be a drawback when dealing with user input, less sensitivity to the prompt is also an issue.

GPT-3.5-0613: The code reading performance bonce of GPT-3.5-0613 in various modifications is noteworthy. The more prompt given leads to better and more stable performance, indicating possible optimization towards complex scenarios in this updated version.

The performance gradients across diverse Prompt Modifications show the strengths and limitations of each model variant. GPT-3.5-0301 works stably in managing different modifications, and GPT-3.5-0613 works alone with accurate prompts, this reveals two models' different focus, and OpenAI's target to let the model has more attention on user input.

2) Implication: The differential performances of the models underscore the evolutionary trajectory of LLMs. As models progress from one iteration to another, the learnings and shortcomings from prior versions play a pivotal role in shaping the next. The robustness of GPT-3.5-0301 in certain scenarios highlights the foundational strengths, while the adaptability of GPT-3.5-0613 showcases the iterative refinements.

Furthermore, the discrepancies in handling Role Definition and Task Declaration modifications provide critical insights into the model's cognitive processes. It emphasizes the importance of context and role comprehension for effective code interpretation.

Despite significant progress, the path to optimizing Large Language Models (LLMs) for code comprehension remains in progress. These insights are valuable for both developers and researchers, helping them in model refinement, training, and adjustment. Furthermore, these findings provide users with a perspective to assess the effectiveness and dependability of various model iterations in practical scenarios.

B. MR2: Code Modification Impact

The primary focus of this research question was to understand how different code modifications impact the code-understanding abilities of LLMs. The results from GPT-3.5-0301 and GPT-3.5-0613 indicate a varying degree of effectiveness based on the modification type. A detailed discussion of these findings will be elaborated upon in this section.

Table III reveals significant differences in the performance of GPT-3.5-0301 and GPT-3.5-0613 across two code modification types, as described before, one is numerical and another is logical. GPT-3.5-0613 consistently outperforms GPT-3.5-0301, particularly excelling in handling Numerical Modifications. This suggests that GPT-3.5-0613 is better equipped to understand arithmetic operations, possibly due to advancements in its training algorithms and data diversity.

TABLE III
IMPACT OF CODE MODIFICATION ON ACCURACY

Modification Target[b]	GPT-3.5-0301 (%)	GPT-3.5-0613 (%)
None	96.29	96.29
Numerical Modification	44.44	49.38
Logical Modification	29.62	30.86

[b]Modification Target is the part of code we conduct modification on. As explained in Chapter III.

1) Numerical Modifications: Numerical modifications typically encompass changes in the arithmetic components of code, including variables, constants, and arithmetic operations. The findings revealed a notable variance in the performance of GPT-3.5-0301 and GPT-3.5-0613 concerning numerical modifications.

GPT-3.5-0613 demonstrated a 5% performance advantage over GPT-3.5-0301, indicating superior code comprehension capabilities when encountering numerical alterations. Several potential reasons account for this disparity:

Improved Training Data: The GPT-3.5-0613 version might have been exposed to a more diverse range of numerical problems during its training phase, leading to a more comprehensive understanding of arithmetic patterns and operations.

Optimized Algorithms: Advancements in the training algorithms of GPT-3.5-0613 could have provided it with a more profound grasp of numerical relationships within code, enabling better performance.

Error Correction Mechanisms: GPT-3.5-0613 might possess improved error correction and feedback loops when processing numerical data, ensuring higher accuracy and reliability.

2) Logical Modifications: Logical modifications generally relate to alterations in the logical constructs of code, such as conditional statements, loops, and Boolean operations. The results indicate a marginal performance improvement in GPT-3.5-0613 over its predecessor.

However, it's noteworthy that the performance difference between the two models concerning logical modifications is minuscule, a mere 1.24% advantage for GPT-3.5-0613. This suggests:

Consistent Logical Understanding: Both versions of the model seem to have a comparable comprehension of logical constructs. This might imply that the foundational training for logical understanding was robust from the outset and has remained consistently effective.

Complexity of Logical Patterns: Logical constructs in code can often manifest in myriad ways, influenced by several interconnected factors. This intrinsic complexity might limit

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 01:36:35 UTC from IEEE Xplore. Restrictions apply.

the scope of significant performance improvements between iterations.

3) Conclusion: The presented data underscores the evolutionary trajectory of LLMs concerning code comprehension capabilities. While GPT-3.5-0613 exhibits marked improvements in numerical understanding, the models maintain a near-par performance in deciphering logical modifications. These findings offer invaluable insights for future model development, emphasizing the need for targeted optimizations and the potential areas of focus.

V. Related Work

The landscape of research on Large Language Models (LLMs) has expanded rapidly, with various studies focusing on their capabilities, limitations, and potential applications. This chapter provides an overview of the most relevant literature in the domains of testing for LLMs and metamorphic testing, highlighting the novelty and originality of our work in comparison.

Testing for Code LLMs has been applied for nearly all existing LLMs, the HumanEval [3] dataset is a Hand-Written Evaluation Set proposed by OpenAI to assess the code generation ability of code LLMs, a wide range of code LLMs, GPT3.5, CodeX, WizardCoder and code-DaVinci-002. Besides, EvalPlus [32] integrated existing benchmarks and added automatically generated test inputs, along with LLM-based strategies, then used for differential tests. MBPP(Mostly Basic Python Programming) [4] including 1,000 entry-level Python programming problems, are widely used for code generation tasks for code generation models.

Metamorphic Testing [5] is a well-established domain in software engineering, primarily used to test software systems that do not have a clear oracle. While traditional metamorphic testing focuses on finding faults in the software, due to the fitness of the method to the machine learning area, more work recently has used this technique in the field of machine learning [28, 29, 30], however, most of them focusing on automating metamorphic testing and improve the readability of LLMs. In the field of Natural Language Processing, Manino [33] proposes three new classes of metamorphic relations to address the systematicity, compositionally, and transitivity of the NLP system. Metamorphic Testing has also been used to evaluate natural language inference models [34].

Our work innovatively applies this concept to evaluate the code-understanding capabilities of LLMs. By introducing to new metamorphic relations in code snippets and prompts to evaluate the models' code reading performance, we provide a fresh perspective on metamorphic testing, adapting its principles for the realm of LLMs.

VI. Conclusion

Our primary objective was to find the code understanding ability of LLMs, specifically GPT-3.5, with prompt and code modification attached to the input. The foundation of our experiment was the metamorphic relations specified for evaluating code understanding ability. We meticulously generated the modification of the HumanEval dataset by introducing new metamorphic relations into code snippets and prompts.

We successfully evaluated the code understanding ability of two variants of GPT-3.5. Our findings provide a clear snapshot of the current capabilities of the models, and the direction of model improvement, revealing both their strengths and areas that require further refinement. GPT-3.5-0613, with its higher weights in user input and possibly richer training data, demonstrated superior performance in rich prompt conditions compared to GPT-3.5-0301, especially in handling complex relational and numerical operators in code and understanding complex situations. We also spotted the weakness of both models having trouble understanding the logical operator modification of the code and numerical value changes in code.

While we've identified potential weaknesses of LLMs in code understanding, vast challenges and potential experiments still await exploration. We will continue to explore prompt engineering using shot-based techniques and will include more programming languages, hoping to explore further software engineering techniques usage in LLMs.

Acknowledgment

Z. thanks Dr. Dongwan Shin (University of Sheffield) for the invaluable discussions and guidance.

References

[1] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann *et al.*, "Palm: Scaling language modeling with pathways," *arXiv preprint arXiv:2204.02311*, 2022.

[2] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[3] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[4] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[5] T. Y. Chen, S. C. Cheung, and S. M. Yiu, "Metamorphic testing: a new approach for generating next test cases," *arXiv preprint arXiv:2002.12543*, 2020.

[6] W. Fedus, I. Goodfellow, and A. M. Dai, "Maskgan: better text generation via filling in the_," *arXiv preprint arXiv:1801.07736*, 2018.

[7] T. Sellam, D. Das, and A. P. Parikh, "Bleurt: Learning robust metrics for text generation," *arXiv preprint arXiv:2004.04696*, 2020.

[8] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry,

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 01:36:35 UTC from IEEE Xplore. Restrictions apply.

A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[9] T. Zhang, F. Ladhak, E. Durmus, P. Liang, K. McKeown, and T. B. Hashimoto, "Benchmarking large language models for news summarization," *arXiv preprint arXiv:2301.13848*, 2023.

[10] J. Pilault, R. Li, S. Subramanian, and C. Pal, "On extractive and abstractive neural document summarization with transformer language models," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 9308–9319.

[11] Z. Jiang, J. Araki, H. Ding, and G. Neubig, "How can we know when language models know? on the calibration of language models for question answering," *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 962–977, 2021.

[12] J. Menick, M. Trebacz, V. Mikulik, J. Aslanides, F. Song, M. Chadwick, M. Glaese, S. Young, L. Campbell-Gillingham, G. Irving *et al.*, "Teaching language models to support answers with verified quotes," *arXiv preprint arXiv:2203.11147*, 2022.

[13] Q. Zheng, X. Xia, X. Zou, Y. Dong, S. Wang, Y. Xue, Z. Wang, L. Shen, A. Wang, Y. Li *et al.*, "Codegeex: A pre-trained model for code generation with multilingual evaluations on humaneval-x," *arXiv preprint arXiv:2303.17568*, 2023.

[14] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago *et al.*, "Competition-level code generation with alphacode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[15] A. Sriram, H. Jun, S. Satheesh, and A. Coates, "Cold fusion: Training seq2seq models together with language models," *arXiv preprint arXiv:1708.06426*, 2017.

[16] A. P. Parikh, O. Täckström, D. Das, and J. Uszkoreit, "A decomposable attention model for natural language inference," *arXiv preprint arXiv:1606.01933*, 2016.

[17] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[19] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," OpenAI, Technical Report, 2018.

[20] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[21] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[22] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[23] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[24] OpenAI, "Gpt-4 technical report," 2023.

[25] F. Cappa, R. Oriani, E. Peruffo, and I. McCarthy, "Big data for creating and capturing value in the digitalized environment: unpacking the effects of volume, variety, and veracity on firm performance," *Journal of Product Innovation Management*, vol. 38, no. 1, pp. 49–67, 2021.

[26] L. Reynolds and K. McDonell, "Prompt programming for large language models: Beyond the few-shot paradigm," in *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems*, 2021, pp. 1–7.

[27] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 824–24 837, 2022.

[28] X. Xie, J. W. Ho, C. Murphy, G. Kaiser, B. Xu, and T. Y. Chen, "Testing and validating machine learning classifiers by metamorphic testing," *Journal of Systems and Software*, vol. 84, no. 4, pp. 544–558, 2011.

[29] A. Dwarakanath, M. Ahuja, S. Sikand, R. M. Rao, R. J. C. Bose, N. Dubash, and S. Podder, "Identifying implementation bugs in machine learning based image classifiers using metamorphic testing," in *Proceedings of the 27th ACM SIGSOFT international symposium on software testing and analysis*, 2018, pp. 118–128.

[30] C. Murphy, G. E. Kaiser, and L. Hu, "Properties of machine learning applications for use in metamorphic testing," 2008.

[31] OpenAI, "Gpt-3.5-turbo-0613: Function calling, 16k context window, and lower prices." 2023, accessed: 2023-06-14. [Online]. Available: https://community.openai.com/t/gpt-3-5-turbo-0613-function-calling-16k-context-window-and-lower-prices/263263

[32] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation," *arXiv preprint arXiv:2305.01210*, 2023.

[33] E. Manino, J. Rozanova, D. Carvalho, A. Freitas, and L. Cordeiro, "Systematicity, compositionality and transitivity of deep nlp models: a metamorphic testing perspective," *arXiv preprint arXiv:2204.12316*, 2022.

[34] M. Jiang, H. Bao, K. Tu, X.-Y. Zhang, and Z. Ding, "Evaluating natural language inference models: A metamorphic testing approach," in *2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2021, pp. 220–230.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 01:36:35 UTC from IEEE Xplore. Restrictions apply.